Northeastern University

EECE5155: Wireless Sensor Networks and the Internet of Things
Laboratory Assignment 2 Report — Colosseum + ns-3 Reconciliation

Author (s):

- Riya Rawal
- Shashank Meghashyam

Date: 10/19/2026

Part 1 — Setup

The Colosseum environment was prepared using a reservation with two Standard Radio Nodes (SRN70 and SRN71) on the webinar-interactive-v1 image. Both nodes were accessed using SSH. RF scenario 1009 was started with automatic cycling and its running status was checked using the Colosseum CLI. This setup was used for subsequent RF emulator and Wi-Fi experiments.
Commands executed: colosseumcli rf start 1009 -c | colosseumcli rf info

Part 2 — Verify the RF emulator (20 points)

Experimental setup

State the scenario you used (1009) and the exact commands/edits you ran. For the retuning questions, show the line(s) you changed in uhd_tx_tone.sh and uhd_rx_fft.sh (e.g., the freq argument).

Results

Provide the spectrum-analyzer screenshots and your analysis, labeled by question.

Q1 — Default tone frequency: 1.01 GHz (1010 MHz)

Experimental setup: With Colosseum RF scenario 1009 running, SRN70 acted as the transmitter and SRN71 as the receiver. The original saved script configurations were restored before running ./uhd_tx_tone.sh (SRN70) and ./uhd_rx_fft.sh (SRN71) from /root/utils. The saved receiver script uhd_rx_fft.sh.bak contains --freq 1.010e9; the restored transmitter also reported 1010 MHz.

Results and analysis: The transmitter reported "Actual TX Freq: 1010.000000 MHz" and the receiver reported "Actual RX Freq: 1010.000000 MHz." The receiver spectrum displayed a strong narrow peak at approximately 1010 MHz, reaching about −27 dBFS against a background noise level of roughly −85 to −100 dBFS. The visible peak confirms that SRN71 detected the tone sent by SRN70 through the Colosseum RF emulator.

Default-frequency verification: The 1.010 GHz setting was recovered from saved pre-retuning scripts, including the receiver backup, and reproduced in both radios. The untouched factory-original script was not independently inspected; the recovered saved setting is reported as the experiment's default.



Figure Q1a. SRN70 transmitter output confirming Actual TX Freq: 1010.000000 MHz after restoring its saved script.

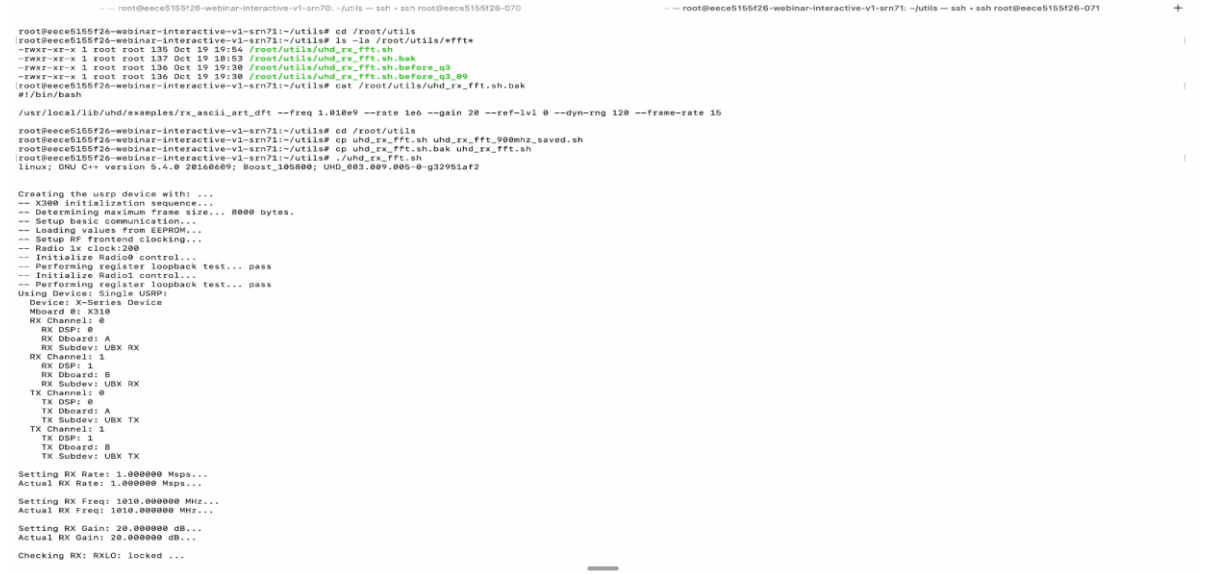


Figure Q1b. SRN71 receiver output confirming Actual RX Freq: 1010.000000 MHz and the saved --freq 1.010e9 setting.

P.T.O

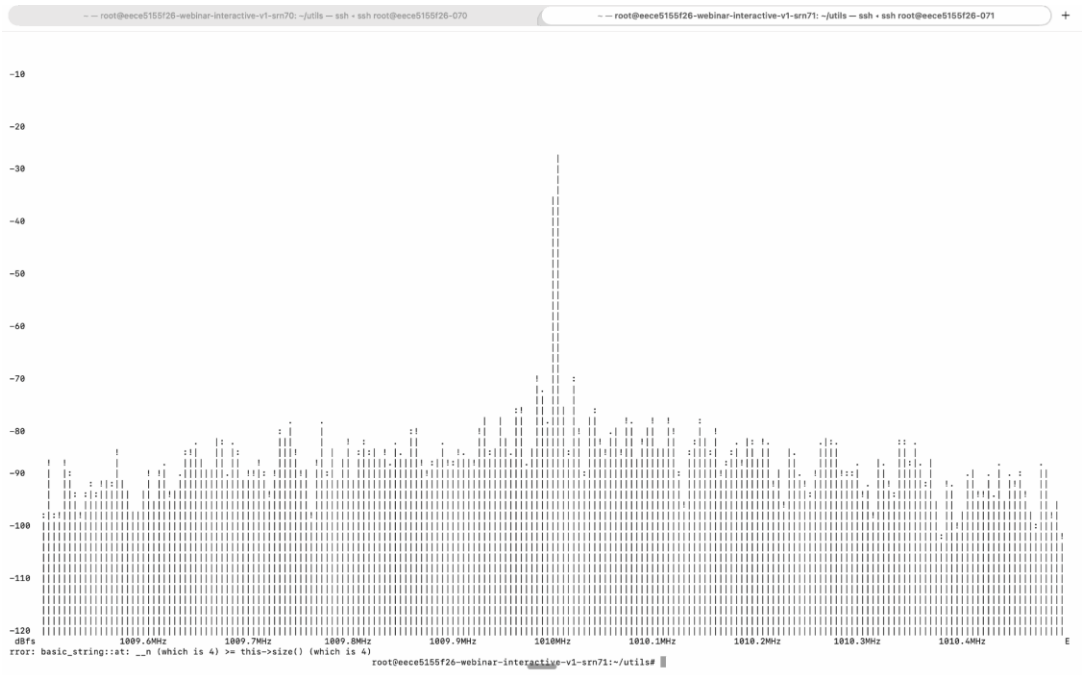


Figure Q1c. SRN71 ASCII spectrum analyzer showing a strong narrow received signal near 1010 MHz.

Q2 — Recorded experiment at 0.99 GHz

Experimental setup: Colosseum RF scenario 1009 was confirmed RUNNING on SRN70. SRN70 was the transmitter and SRN71 the receiver. Both scripts were retuned to 0.99 GHz (990 MHz), then ./uhd_tx_tone.sh and ./uhd_rx_fft.sh were run from /root/utils.
Results: The transmitter reported Actual TX Freq: 990.000000 MHz and Actual TX Gain: 20.000000 dB. The receiver reported Actual RX Freq: 990.000000 MHz and Actual RX Gain: 20.000000 dB. The receiver spectrum shows a prominent narrow peak at approximately 990 MHz against a lower noise floor, supporting successful reception of the retuned tone.



Figure Q2a. SRN70 transmitter terminal at 0.99 GHz (990 MHz), including the TX frequency confirmation.

P.T.O

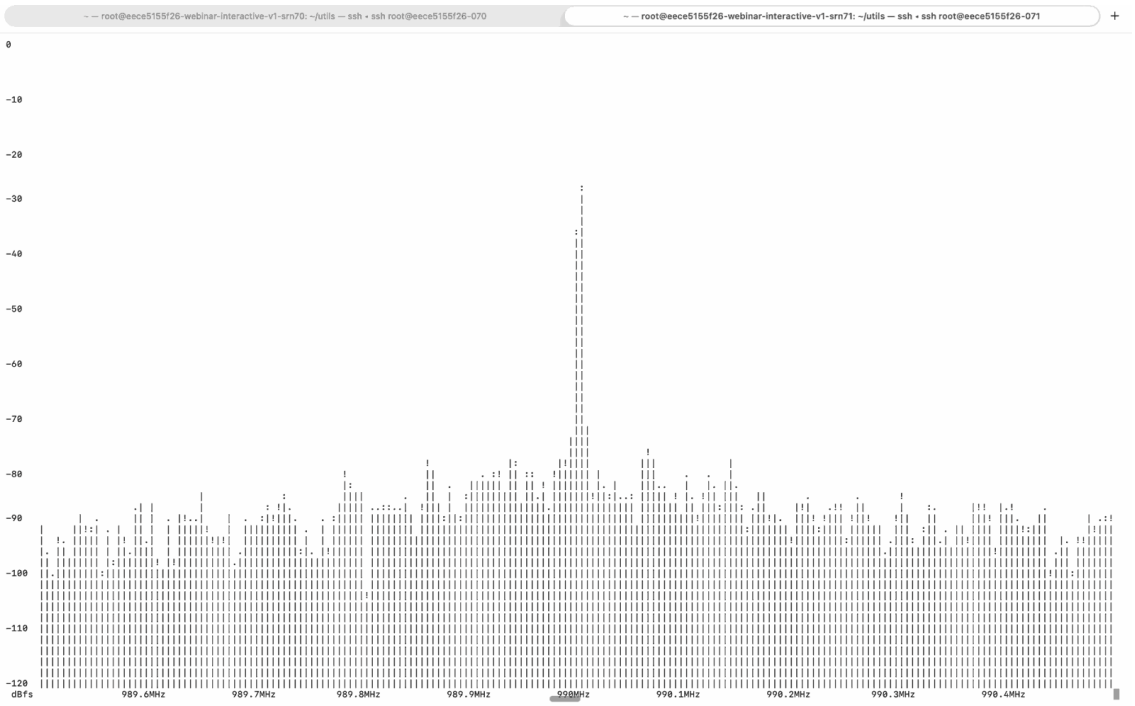


Figure Q2b. SRN71 receiver spectrum analyzer showing the received peak near 990 MHz.

```
linux; GNU C++ version 5.4.0 20160609; Boost_105800; UHD_003.009.005-0-g32951af2


Creating the usrp device with: ...
-- X300 initialization sequence...
-- Determining maximum frame size... 8000 bytes.
-- Setup basic communication...
-- Loading values from EEPROM...
-- Setup RF frontend clocking...
-- Radio 1x clock:200
-- Initialize Radio0 control...
-- Performing register loopback test... pass
-- Initialize Radio1 control...
-- Performing register loopback test... pass
Using Device: Single USRP:
  Device: X-Series Device
  Mboard 0: X310
  RX Channel: 0
    RX DSP: 0
    RX Dboard: A
    RX Subdev: UBX RX
  RX Channel: 1
    RX DSP: 1
    RX Dboard: B
    RX Subdev: UBX RX
  TX Channel: 0
    TX DSP: 0
    TX Dboard: A
    TX Subdev: UBX TX
  TX Channel: 1
    TX DSP: 1
    TX Dboard: B
    TX Subdev: UBX TX

Setting TX Rate: 0.100000 Msps...

UHD Warning:
    The hardware does not support the requested TX sample rate:
    Target sample rate: 0.100000 MSps
    Actual sample rate: 0.390625 MSps

UHD Warning:
    The hardware does not support the requested TX sample rate:
    Target sample rate: 0.100000 MSps
    Actual sample rate: 0.390625 MSps
Actual TX Rate: 0.390625 Msps...

Setting TX Freq: 990.000000 MHz...
Actual TX Freq: 990.000000 MHz...

Setting TX Gain: 20.000000 dB...
Actual TX Gain: 20.000000 dB...

Setting device timestamp to 0...
Checking TX: TXLO: locked ...
Press Ctrl + C to stop streaming...
^C
Done!

root@eece5155f26-webinar-interactive-v1-srn70:~/utils#
```

Figure Q2c. SRN70 transmitter terminal confirms Actual TX Freq 990 MHz and completion of tone transmission.

P.T.O



Figure Q2d. SRN71 shows the edited uhd_rx_fft.sh frequency argument and confirms Actual RX Freq 990 MHz.

Verified frequency-setting script lines (as shown in the SRN70 and SRN71 terminal screenshots):
SRN70 — uhd_tx_tone.sh: /usr/local/lib/uhd/examples/tx_waveforms --freq 0.99e9 --rate 1e5 --gain 20
SRN71 — uhd_rx_fft.sh: /usr/local/lib/uhd/examples/rx_ascii_art_dft --freq 0.99e9 --rate 1e6 --gain 20 --ref-lvl 0 --dyn-rng 120 --frame-rate 15
On SRN71, the captured terminal shows editing the previous --freq 1.010e9 value to --freq 0.99e9, followed by execution and confirmation of the receiver setting. UHD noted a transmitter sample-rate adjustment from the requested 0.100000 Msps to an actual 0.390625 Msps; this did not prevent the 990 MHz tone from being observed.

Experimental setup — Q3

Using RF scenario 1009, SRN70 (transmitter) and SRN71 (receiver) were both retuned to 0.900 GHz (900 MHz). The scripts uhd_tx_tone.sh and uhd_rx_fft.sh were executed from /root/utils. The verified --freq argument was 0.9e9 on both nodes.

SRN70 transmitter: /usr/local/lib/uhd/examples/tx_waveforms --freq 0.9e9 --rate 1e5 --gain 20
SRN71 receiver: /usr/local/lib/uhd/examples/rx_ascii_art_dft --freq 0.9e9 --rate 1e6 --gain 20 --ref-lvl 0 --dyn-rng 120 --frame-rate 15

Results — Q3

The SRN70 terminal reports “Actual TX Freq: 900.000000 MHz,” and the SRN71 terminal reports “Actual RX Freq: 900.000000 MHz.” The received spectrum is centered on 900 MHz but has no distinct narrow tone; displayed levels are largely around −90 to −100 dBFS, unlike the prominent peak at 990 MHz recorded in Q2. Colosseum scenario 1009 is centered at 1000 MHz (1.000 GHz) and specifies 80 MHz usable transmission bandwidth. Its usable frequency interval is 1000 ± 40 MHz, or 960–1040 MHz (0.960–1.040 GHz). A 900 MHz tone is 60 MHz below the lower passband edge, explaining why no clear received tone appears in this

P.T.O

measurement. Source: Colosseum documentation, “All Paths 0 dB (1009),” https://colosseumwireless.readthedocs.io/en/latest/scenarios/test_scenarios.html
Note on receiver troubleshooting: The SRN71 capture also includes a later attempt to restart the receiver that returned “No devices found.” This subsequent USRP device-access error does not invalidate the earlier successful frequency readout, but the missing tone should be interpreted from the spectrum captured while the receiver was operating, not from the failed restart.



Figure Q3a. SRN70 transmit terminal showing --freq 0.9e9 and actual TX frequency of 900 MHz.



Figure Q3b. SRN71 receive terminal showing --freq 0.9e9 and actual RX frequency of 900 MHz (with later USRP access error).

P.T.O

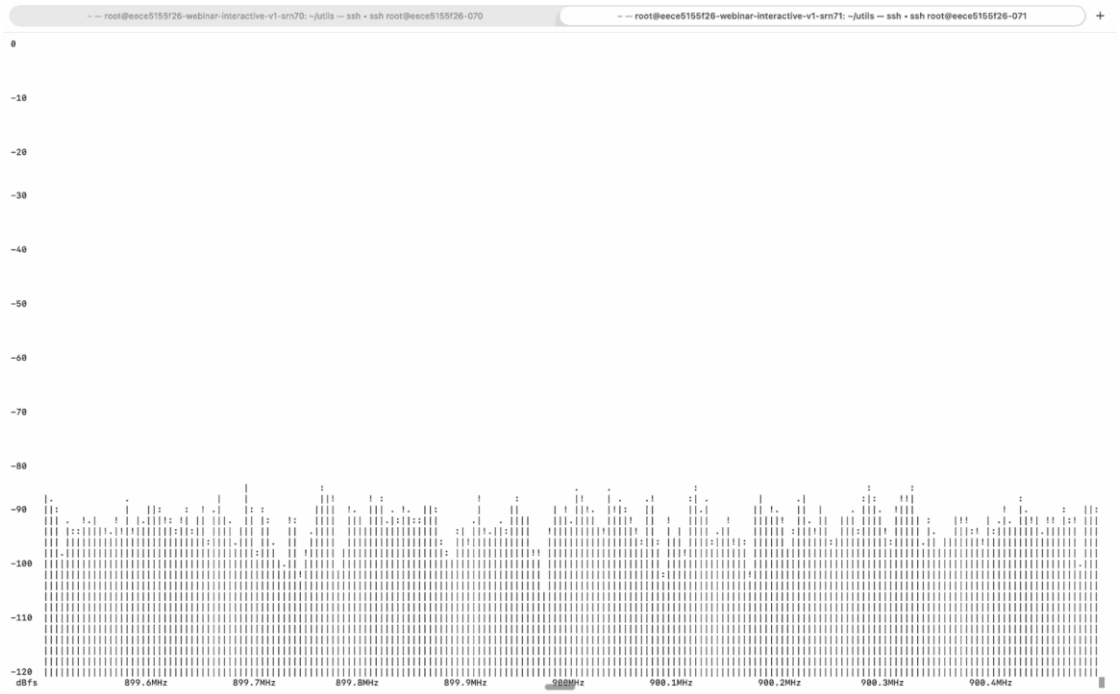


Figure Q3c. SRN71 spectrum centered on 900 MHz with no clear received tone above the noise floor.

Part 3 — Wi-Fi ad hoc network and measurements (25 points)

Experimental setup

List the tap_setup.sh / route_setup.sh / modem_start.sh commands you ran (with the SRN ID you used), and the ping and iperf3 commands.

Colosseum RF scenario 1009 was started from SRN70 with `colosseumcli rf start 1009 -c` (start time 19:05:41) and checked with `colosseumcli rf info`. SRN70 and SRN71 were configured as a two-node Wi-Fi ad hoc network. On each node, [tap_setup.sh] created the tap0 interface; modem_start.sh launched the Wi-Fi transceiver on the USRP, and route_setup.sh, given the peer's SRN ID, added a route to the peer's internal subnet through tap0.

	SRN70	**SRN71**
tap0 address (MAC)	172.20.22.170 (aa:bb:cc:dd:ee:aa)	172.20.22.171 (aa:bb:cc:dd:ee:ab)
tr0 address	192.168.170.1	192.168.171.1

Commands run on SRN70:

```
[/root/interactive_scripts/tap_setup.sh ...]
/root/interactive_scripts/modem_start.sh
[/root/interactive_scripts/route_setup.sh 171]
ping -c 30 192.168.171.1
iperf3 -s  (iperf3 server)
```

Commands run on SRN71:

```
[/root/interactive_scripts/tap_setup.sh ...]
/root/interactive_scripts/modem_start.sh
/root/interactive_scripts/route_setup.sh 170
```


P.T.O

```
ping -c 30 192.168.170.1
iperf3 -c 172.20.22.170 -u -b 5M  (iperf3 client, 5 Mbps UDP)
```

Note: Earlier 5-packet ping attempts from both nodes were made before the link was operational (on SRN70, tap0 showed NO-CARRIER and the RF scenario was not yet running) and failed with "Destination Host Unreachable" (100% loss). After scenario 1009 was confirmed RUNNING, a duplicate route on SRN70 pointing to its own subnet was removed with ip route del 192.168.170.0/24 via 172.20.22.170 dev tap0, and the measurements were repeated. Rerunning route_setup.sh 170 on SRN71 returned "SIOCADDRT: File exists" because the route was already in place. For iperf3, the first client attempt from SRN71 returned "Connection refused" because the server was not yet running. The next two runs used iperf3 3.0.11 on SRN71 against iperf3 3.6+ on SRN70; after running ldconfig on SRN71, its iperf3 reported version 3.6+, matching SRN70, and the run reported in Q6 was made with matching versions.

Results: Provide the screenshots (modem output, ping summary, iperf3 output) and your analysis, labeled by question.

Q4 — messages exchanged after modem_start.sh (screenshot + comment)

Experimental setup: SRN70 and SRN71 ran /root/interactive_scripts/modem_start.sh, which initialized each node's X300 USRP and allocated the virtual Ethernet interface tap0. SRN70's tap0 is 172.20.22.170 (MAC aa:bb:cc:dd:ee:aa) and SRN71's tap0 is 172.20.22.171 (MAC aa:bb:cc:dd:ee:ab). With the modem running, tap0 was cycled down/up while tcpdump -i tap0 -n -e -vv ip6 captured the IPv6 traffic on each node.

Results and analysis: After /root/interactive_scripts/modem_start.sh was run on SRN70 and SRN71, each X300 USRP completed its initialization (register loopback tests passed on Radio0 and Radio1) and the modem allocated the virtual Ethernet interface tap0, which connects the Linux network stack to the Wi-Fi transceiver. The modem then printed a line for each Ethernet frame it handled. Most appear as "unknown ether type"; tcpdump on tap0 identifies these as IPv6 control frames (ethertype 0x86DD) that each node generates automatically when the interface comes up: MLDv2 listener reports to ff02::16, a neighbor solicitation from :: for the node's EUI-64 link-local address (Duplicate Address Detection, e.g., fe80::a8bb:ccff:fedd:eeab on SRN71), and router solicitations to ff02::2 repeated at increasing intervals (about 4, 8 and 16 s). No router advertisements were received, which is consistent with an ad hoc network that has no router. The SRN71 modem also logged "ether type: ARP" frames, the IPv4 address-resolution messages that must complete before ICMP or UDP traffic can be sent to the peer. The occasional "L" characters are UHD late-packet indicators on the transmit path.

Note on scope: Each tcp dump capture shows only that node's own outgoing frames, so these screenshots document the messages generated after modem startup, not a completed two-way exchange.

P.T.O



Figure Q4a. SRN71 modem_start.sh output: X300 initialization, tap0 allocation, and the frames handled by the modem ("unknown ether type" IPv6 frames and "ether type: ARP" frames)



Figure Q4b. SRN70 modem_start.sh output showing the same initialization and tap0 allocation. The ping above it was run before the modem and RF scenario were active (tap0 NO-CARRIER) and is not used as a Q5 result.

P.T.O

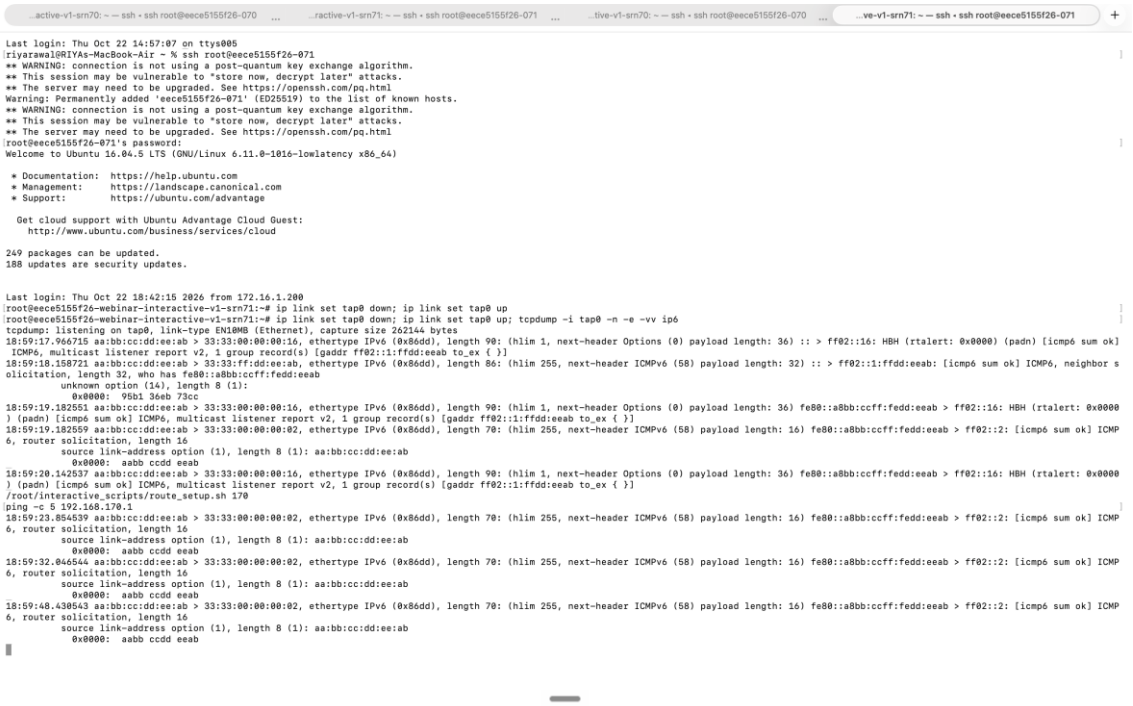


Figure Q4c. tcpdump on SRN71 tap0 identifying the IPv6 frames: MLDv2 listener reports, neighbor solicitation for Duplicate Address Detection, and router solicitations.

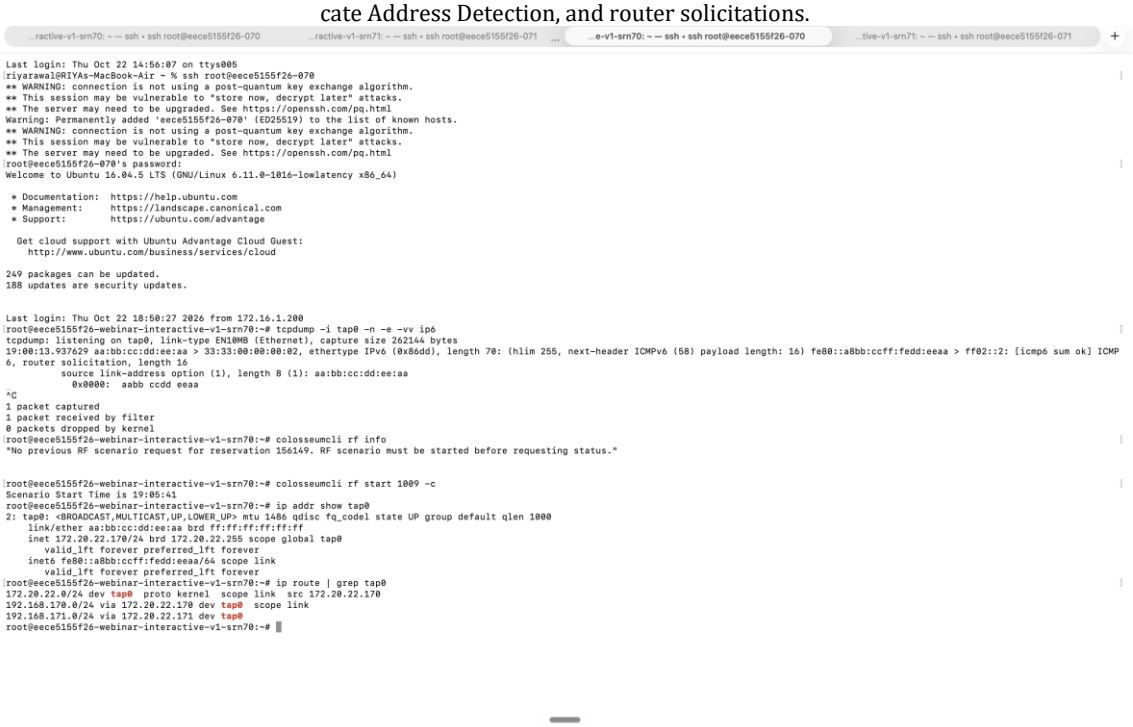


Figure Q4d. tcpdump on SRN70 tap0 showing a router solicitation from fe80::a8bb:ccff:fedd:eeaa, followed by the RF scenario start and the tap0 address and routes.

P.T.O

Q5 — average RTT, one-way delay (state assumption), packet loss

Experimental setup: With scenario 1009 RUNNING and both modems active, 30 ICMP echo requests were sent in each direction over the Wi-Fi link: ping -c 30 192.168.171.1 from SRN70 and ping -c 30 192.168.170.1 from SRN71.

Results: Table Q5 summarizes both ping runs.

Table Q5. Ping statistics over the Colosseum Wi-Fi ad hoc link (30 packets per direction).

Measurement	**SRN70 → SRN71 (192.168.171.1)**	**SRN71 → SRN70 (192.168.170.1)**
Packets transmitted / received	30 / 30	30 / 30
Packet loss	0%	0%
RTT min / avg / max	14.647 / 15.100 / 16.990 ms	14.685 / 14.881 / 15.678 ms
RTT mdev (jitter)	0.575 ms	0.241 ms
One-way delay (RTT/2)	≈ 7.55 ms	≈ 7.44 ms

Average RTT: 15.100 ms from SRN70 and 14.881 ms from SRN71, or about 15.0 ms overall.

One-way delay: about 7.5 ms (7.55 ms and 7.44 ms). Assumption: the link is symmetric, so the forward and return paths take the same time and one-way delay = RTT / 2. The time the far node takes to process the echo request and send the reply is assumed negligible compared with the RTT.

Packet loss: 0% in both directions (30/30 received at each node).

Observations: The two directions agree closely (15.100 ms vs. 14.881 ms, a difference of about 0.2 ms), which supports the symmetry assumption. Jitter is low (mdev 0.575 ms and 0.241 ms), so the delay is steady from packet to packet. The first packets were slower: 16.9 ms for SRN70's first echo, and 32.4 ms in SRN71's earlier 5-packet test (ping -c 5 192.168.170.1, RTT avg 18.182 ms), consistent with ARP resolution before the first echo could be sent. After that, RTT settles at about 14.7–14.9 ms.

Values carried RTT ≈ 15.0 ms; one-way delay ≈ 7.5 ms.

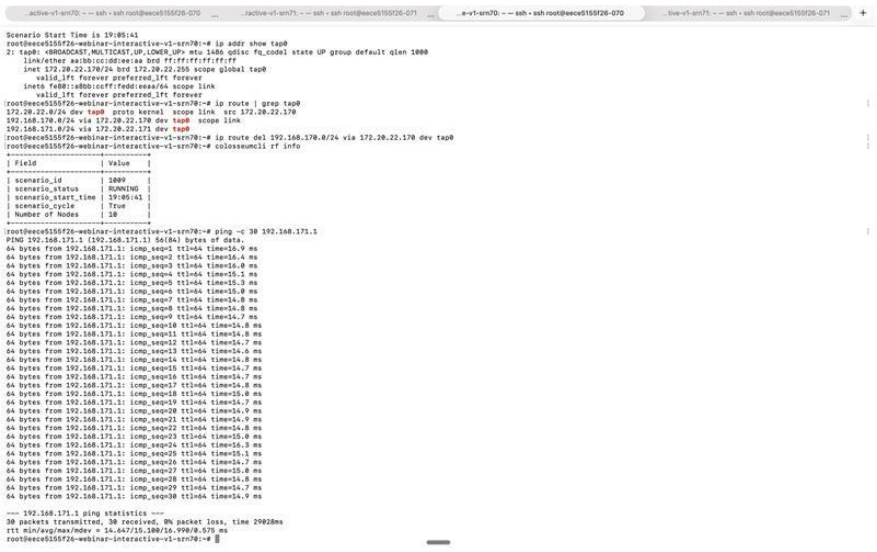


Figure Q5a. SRN70: scenario 1009 RUNNING, removal of the duplicate route, and ping -c 30 192.168.171.1 (30/30 received, 0% loss, RTT avg 15.100 ms, mdev 0.575 ms).

P.T.O

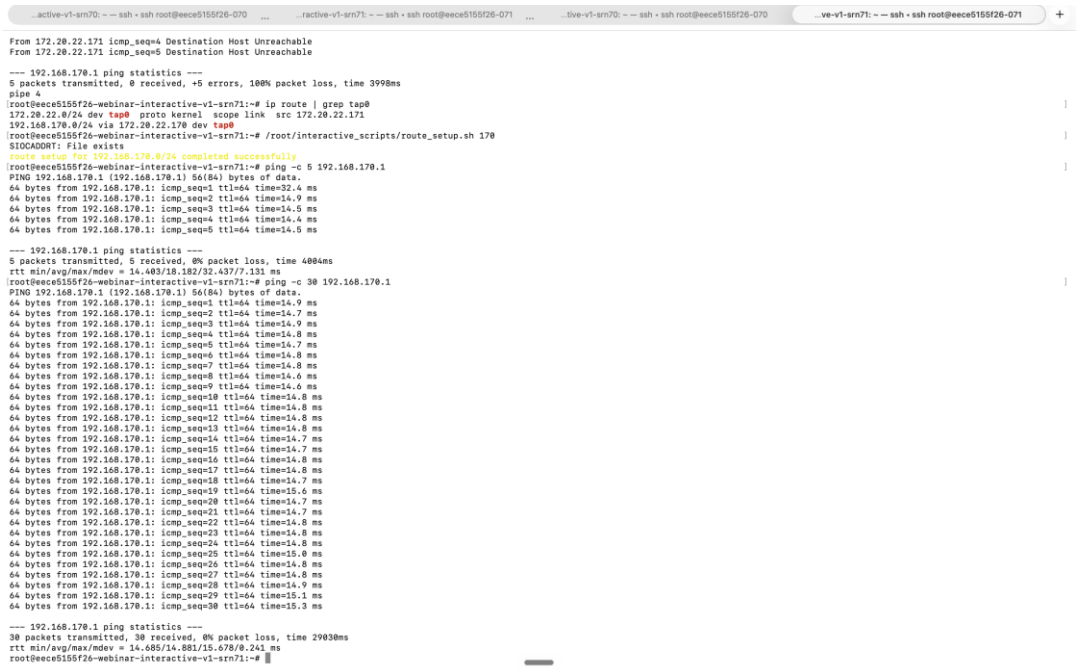


Figure Q5b. SRN71: route_setup.sh 170, ping -c 5 and ping -c 30 192.168.170.1 (30/30 received, 0% loss, RTT avg 14.881 ms, mdev 0.241 ms).

Q6 — iperf3 delivered throughput vs the 5 Mbps offered (record this number for Part 5)

Experimental setup: SRN70 ran the iperf3 server (iperf3 -s) and SRN71 ran the client, offering 5 Mbps of UDP traffic to SRN70's tap0 address (iperf3 -c 172.20.22.170 -u -b 5M, default 10 s duration). Both nodes used iperf3 3.6+ for the reported run (client port 53375).

Results: Table Q6 summarizes the sender and receiver reports. Neither side printed a final summary line: the client stopped after 9 one-second intervals with "control socket has closed unexpectedly," and the server ended with "select failed: Bad file descriptor." The delivered throughput is therefore taken from the server's per-interval reports.

Table Q6. iperf3 UDP test, SRN71 → SRN70, 5 Mbps offered.

Measurement	Client (SRN71, sender)	Server (SRN70, receiver)
Offered / sent bitrate	5.00 Mbps (4.99–5.00 Mbps per interval)	—
Delivered bitrate	—	2.06–2.23 Mbps per interval; mean ≈ 2.11 Mbps
Datagrams per second	≈ 436	≈ 180–194
Datagrams sent / counted	≈ 3,923 (9 reported intervals)	2,576 (14 reported intervals)
Jitter	—	2.79–3.03 ms
Lost / total (iperf3 counter)	—	0 (0%) in every interval

Delivered throughput vs. offered: about 2.11 Mbps delivered against 5 Mbps offered, or roughly 42% of the offered load. The server's per-interval bitrate was steady (2.06–2.23 Mbps), and the total received data (3,607 KBytes over 14 s) gives the same 2.11 Mbps.

Observations: The client sent about 436 datagrams per second at 5.00 Mbps, while the server counted only about 180–194 per second. The server reported 0 lost datagrams because there were no gaps in the

P.T.O

sequence numbers it received, so the 0% figure does not mean all offered traffic was delivered: about 2,576 of roughly 3,923 sent datagrams were counted at the server. Datagrams continued to arrive for about 5 s after the client stopped sending (server intervals run to 14 s). Jitter at the receiver was about 3 ms.

Earlier runs with mismatched iperf3 versions: With iperf3 3.0.11 on the client (about 76 datagrams per second), the server received 1.64–1.97 Mbps per interval (mean ≈ 1.85 Mbps) with 3–16% datagram loss and about 28–45 ms jitter. These runs are shown for completeness and are not used as the Q6 result.

Value carried to Part 5: delivered throughput ≈ 2.11 Mbps of 5 Mbps offered.

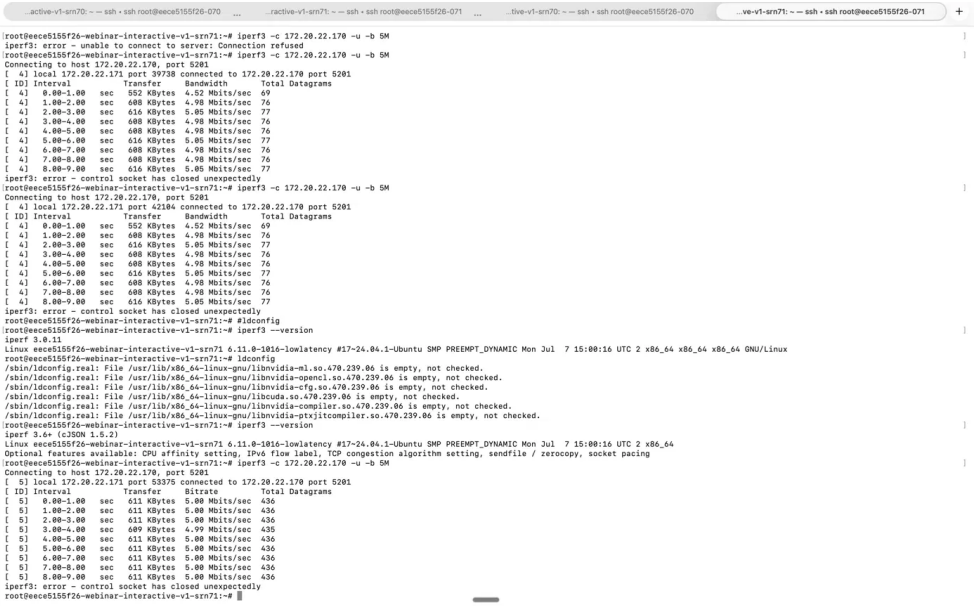


Figure Q6a. SRN71 iperf3 client (iperf3 -c 172.20.22.170 -u -b 5M): earlier runs with iperf3 3.0.11, ldconfig, and the reported run with iperf3 3.6+ sending 5.00 Mbps (about 436 datagrams/s).

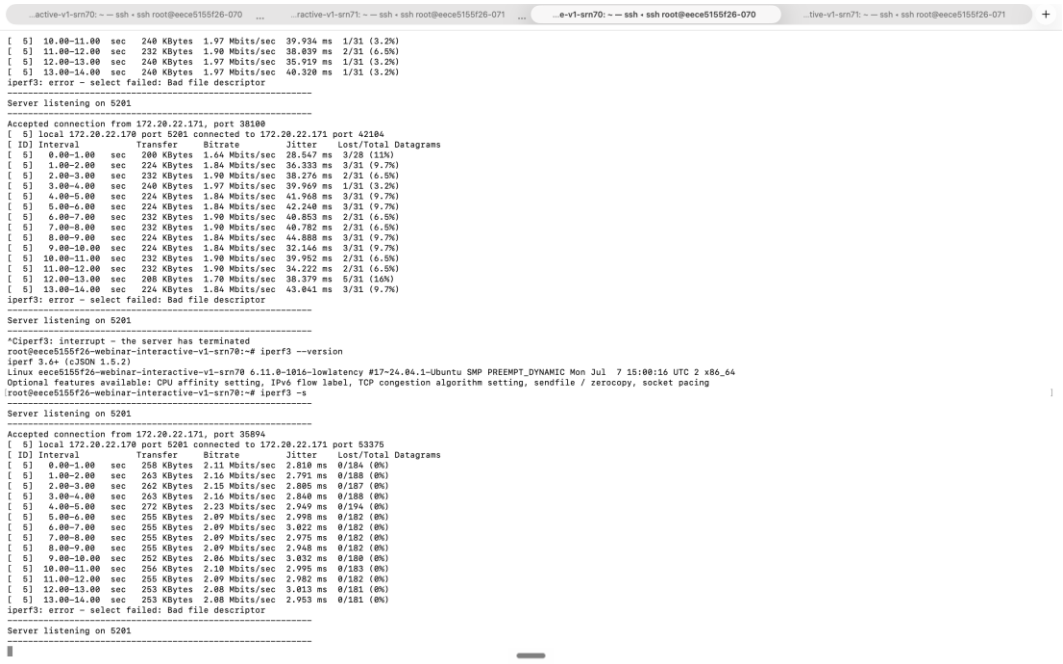


Figure Q6b. SRN70 iperf3 server (iperf3 -s): earlier runs (1.64–1.97 Mbps, 3–16% loss) and the reported run (bottom) receiving 2.06–2.23 Mbps with about 3 ms jitters.

P.T.O

Part 4 — Equivalent ns-3 model (20 points)

Experimental setup

Describe the two-node ns-3 scenario in lab2-wifi-adhoc.cc and the command you ran. Here the "explain the relevant lines of code" guidance applies point to the lines that set the 802.11 standard, the ad hoc MAC, the offered UDP rate, and the FlowMonitor instrumentation. For example,

ns3 run scratch/lab2-wifi-adhoc

Two stationary nodes were configured in ns-3.48 as an IEEE 802.11g ad hoc Wi-Fi network with a default separation of 10 m. The Yans Wi-Fi PHY/channel helpers used the supplied default channel configuration; fixed locations used ConstantPositionMobilityModel. A single UDP application sent from node 0 (10.0.0.1) to node 1 (10.0.0.2), using a configured offered rate of 5 Mbps and 1400-byte packets. The sender operated from simulation time 1.0 s to 10.0 s, while the sink started at 0.0 s. FlowMonitor collected transmitted/received packet counts, delivered throughput, and mean packet delay.
Command executed from the ns-3.48 root: ./ns3 run scratch/lab2-wifi-adhoc

Relevant source-code statements:

```
wifi.SetStandard (WIFI_STANDARD_80211g);                    // IEEE 802.11g
mac.SetType ("ns3::AdhocWifiMac");                          // ad hoc MAC
onoff.SetConstantRate (DataRate (offeredRate), 1400);     // 5 Mbps, 1400-byte payload
Ptr<FlowMonitor> monitor = fmHelper.InstallAll ();        // monitor both nodes
```

Results:

Report the ns-3 delivered throughput and mean delay, and present the ns-3 vs Colosseum numbers side by side in a small table (offered rate, delivered throughput, latency).

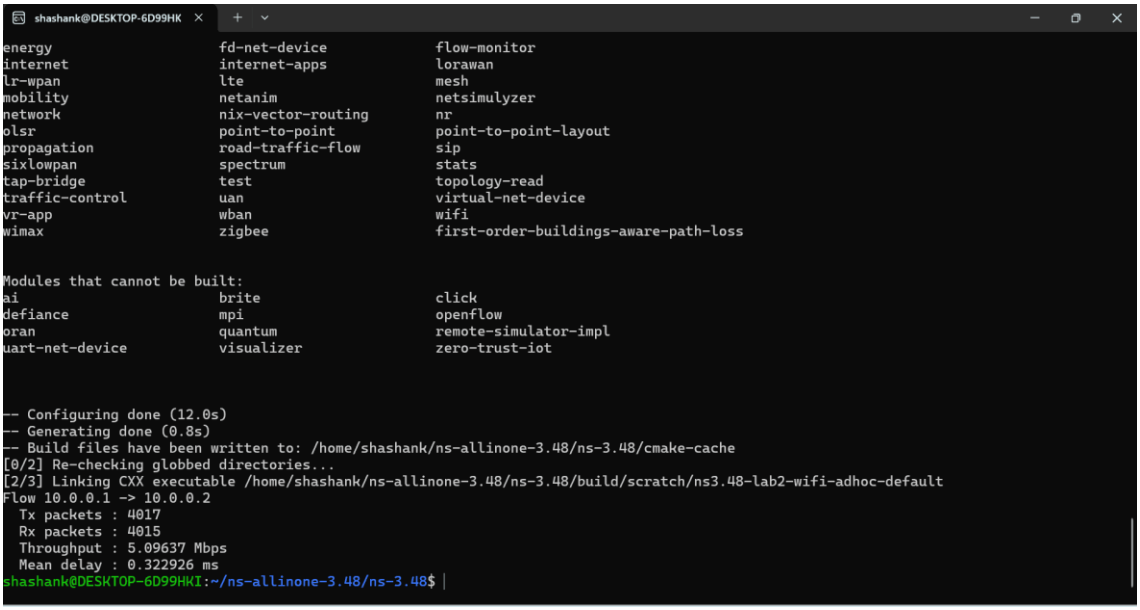


Figure 7. Successful ns-3.48 execution and Flow Monitor output (screenshot taken during the experiment).

P.T.O

Observed ns-3 simulation output:

Flow: 10.0.0.1 → 10.0.0.2
Transmitted packets: 4,017
Received packets: 4,015
Delivered throughput: 5.09637 Mbps
Mean end-to-end delay: 0.322926 ms

Comparison table — ns-3 simulation vs. Colosseum experiment

Measurement	ns-3 simulation	Colosseum experiment
Offered UDP rate	5 Mbps	5 Mbps
Delivered throughput	5.09637 Mbps	≈ 2.11 Mbps (mean of iperf3 server per-interval reports, SRN71→SRN70)
Latency	0.322926 ms mean one-way flow delay	15.100 ms mean RTT (SRN70→SRN71); about 7.5 ms one-way, assuming a symmetric link (RTT/2)

Quantitative comparison: Both experiments offered the same 5 Mbps UDP load. The numerical differences are:

Delivered throughput: ns-3 delivered 5.09637 Mbps, while Colosseum delivered about 2.11 Mbps, a difference of about 2.99 Mbps. The Colosseum throughput is about 41% of the ns-3 value and about 42% of the offered 5 Mbps; the ns-3 value is about 102% of the offered rate.

Latency: ns-3 reported a mean one-way flow delay of 0.322926 ms. Colosseum gave about 7.5 ms one-way (RTT/2 of the 15.100 ms SRN70→SRN71 mean RTT), about 7.2 ms more, or roughly 23 times the ns-3 value.

Packet delivery: ns-3 received 4,015 of 4,017 transmitted packets (2 lost, about 0.05%). In Colosseum, ping lost 0 of 30 packets in each direction, while the iperf3 server counted about 2,576 of the roughly 3,923 UDP datagrams the client sent (about 66%).

Note comparability: The two latency values are not the same metric. The ns-3 figure is the FlowMonitor mean one-way delay of the 5 Mbps UDP flow; the Colosseum figure is derived from ICMP ping RTT measured separately from the iperf3 test, halved under the symmetric-link assumption stated in Q5.

Part 5 — Reconcile the results: audit the AI explanation (35 points)

Claim A: "The wireless channel is the cause of the major discrepancy between the results of ns-3 and Colosseum simulations, since they use different channels." Claim A states: "Since they do not have identical channels, the wireless channel causes the major discrepancy between the ns-3 and Colosseum simulations."

Verdict and Conclusions: The conclusion that can be drawn from this experiment is that the wrong explanation was produced. Scenario 1009 is named "All Paths 0 dB." Again, for the same reason, it does not take the contribution of Colosseum hardware into account, but the single-tap channel with the modeled path loss of 0 dB is defined. Therefore, the only factor that could be taken into account in this scenario is distance-dependent attenuation and realistic fading, which cannot explain the discrepancy.

Corrected statement: To investigate the discrepancy, the assumptions used in the simulation should be compared with the real implementation of the radio concerning processing, buffering, the influence on

P.T.O

hardware, and PHY/MAC layer configuration. The path-loss attenuation or fading in the scenarios cannot explain scenario 1009.

Claim B: "In ns-3 the two nodes inter-communicate via an idealized channel whereas in Colosseum Massive Channel Emulator there is realistic multipath fading and path loss attenuation between the SRNs."

Verdict and Conclusions: ns-3 may be deemed to be idealized, but the description of scenario 1009 is flawed. Ns-3 does not use the signal processing as seen in real SDR which could even be considered idealized; however, this is not seen as an absence of any propagation loss or propagation delay. In our report, the script is setup for the default channel, which includes propagation delay and log-distance propagation loss. The statement about Colosseum of the most complex possible channel in the selected scenario is not correct as the emulator is running the selected scenario (not the most complex one) The explanation indicates that there should be no realistic multipath fading and attenuation added in Scenario 1009, which is a constant single tap channel with 0 dB channel gain. The documentation is kept aside for effects related to hardware.

Corrected statement: While in ns-3 there is an abstract WiFi physical layer as well as channel, in Colosseum a real radio is used in the modeled-loss channel with a constant loss of 0 dB for scenario 1009. The variation is not in the hostility of the propagation surroundings brought on by the propagation.

Claim C — "This added attenuation lowers the received SNR, which forces the 802.11 rate-adaptation algorithm down to a lower data rate and triggers retransmissions; that is what reduces the throughput … down to about 0.64 Mbps."

Verdict and Conclusions: This is not a causation relationship proven. It may be possible that physical causality exists in general, but there is no proof. Affects may reduce the signal to noise ratio (SNR) of the thereby cause poor reception that leads to the change of the rate. But for 1009 scenario, the additional model of attenuation has not been taken into account. There was no proof about the presence of the adaptive rate algorithm on the radio, whether the reduction of the rate was caused, or the reason behind the reduction was the increase in the number of retries. Details of rates and configurations and number of retries should be specified. There is also a possibility that the hardware will be lost, but at this time, there is no sufficient evidence. In addition, our results do not support statistics. Rather than 0.64 Mbps, we were offered 5 Mbps and achieved 2.11 Mbps. This number has been retrieved from the interval report of the server. The iperf3 test failed and could not produce the typical iperf3 final report.

Correct Statement: Our observations show that throughput is low, and the reasons are unknown at this time. If there is evidence from the measurements, only rates fall back and retransmissions should be included in the reasons. In addition, the processing capacity, the buffer capacity, the differences in configuration, and the incompleteness of the measure must be considered.

Claim D: "The roughly 14ms round-trip time is the propagation delay of the signal travelling the emulated distance between the two nodes, which the ns-3 default channel does not reproduce."

Verdict and Conclusions: Both the characterization of the RTT and the default in ns-3 are wrong. The location and separation range between the SRN for Scenario 1009 have not been specified. So, there's no emulated distance to be assumed where it is being measured that the RTT is a millisecond. The measured RTT is as described above; it

includes the complete path of request to reply, rather than just propagation delay. The measured RTT may also be delayed by real processing and buffering, such as GNU Radio, that has delayed its processing and buffering. The propagation delay is used by default in Yans Wi-Fi model using a constant speed propagation model in ns-3. What is not included, however, is the latency of execution of the SDR software/hardware pipeline, which is nsnam+1.

Correct Statement: The measured RTT values for the Colosseum are 15.100 ms and 14.881 ms, respectively, which would give an approximate one-way delay (based on the symmetry assumption made) of 7.5ms. The ns-3 RTT value of 0.322926ms is not the half of the ICMP measured RTT but is the mean of one-way UDP flow delay. The RTT is not the distance propagation time, but rather the end-to-end latency time. The difference between ns-3 may be the overhead that was not captured in the simulations; however, the individual components of this overhead need to be measured.

Claim E: "In short, ns-3 overestimates performance because it ignores the physical propagation effects that Colosseum models."

Verdict and Conclusions: Both its description and its explanation of the experiment is incorrect. Propagation effects are not ignored in ns-3 because the default Yans model takes into account both loss and propagation delay. Scenario 1009 does not implement a propagation-based channel, but a constant channel as claimed. The main key difference between the two systems is that of hardware-in-the-loop emulator versus discrete event simulator. Ns-3 simulates events using the simulation clock; the computer time necessary to process the event does not automatically translate to delay in packet processing. The delay will have to be Designed. In reality, processing and buffering time contribute to the time taken. nsnam.

Correct Statement: A simpler ns-3 model will predict higher throughput and lower delays as the actual implementation may not be constrained by the processing and buffering delays, because ns-3 doesn't have propagation models per se.

Claim F: "Adding a realistic propagation-loss and fading model to the ns-3 script would therefore close the gap almost completely."

Verdict and Conclusions: The simulation will yield a lower throughput than the real throughput, and the results will be more realistic when compared to the experiment, but it will not reflect the real scenario. The simulation has not modeled other parameters like buffering and processing time quite to perfection.

Corrected statement: Apply the channel matching the specified scene 1009, taking into account parameters of the hardware influences and of the traffic and PHY/MAC influences. Develop a model that can be used to measure delay and capacity constraints of measure implementation. There is no available evidence to suggest that the fading procedure will be successful in remediating the problem. After the correction there is an explanation of the difference. For the case of scenario 1009 "All Paths 0 dB", which is a constant single-tap channel with no extra path loss model other than hardware effects, fading or attenuation caused by distances cannot explain this difference. Results from ns-3 run show the average throughput to be 5.09637 Mbps and for UDP, one way delay to be 0.322926 ms, which is significantly lower as compared to Colosseum, where it is ~ 2.11 Mbps and ping RTT ~ 15ms (preliminary due to errors in iperf3 run) respectively. The time as simulated in the simulation in ns-3 is using a modelling approach and hence can be avoided in the real world implementation of SDR as there are process, buffer, and some overheads in the hardware-path related to these.

P.T.O

Explanation of the discrepancy after corrections:

Scenario 1009 “All Paths 0 dB” is defined to be a constant single-tap channel with no additional path loss model applied other than hardware effects, therefore, fading or attenuation due to distances cannot explain the discrepancy. The ns-3 run shows throughput equal to 5.09637 Mbps and UDP one-way delay of 0.322926 ms, while Colosseum showed around 2.11 Mbps throughput and ping RTT of 15 ms. The Colosseum throughput value is preliminary as there were errors in iperf3 run, while latencies have been obtained using different methods. The main difference here is that the ns-3 simulator uses the modeling approach for advancing the simulation time, while the real-world implementation of SDR involves some overhead due to processing, buffering, and hardware-path operations which could be omitted in the model.

Appendix: AI use

Complete one block per tool used. If you did not use any AI tools, write "None".

- Tool and version:
- What I used it for:
- Prompts (or a faithful summary):
- What I verified or changed before including the output:

Tool and version: ChatGPT (GPT-6).

What I used for: Linux and ns-3 guidance, drafting and formatting Parts one through four, describing the user provided Colosseum screenshots, and incorporating questions one through three evidence into the report. No AI assistance was used for Part 5, Q8.

Prompts faithful summary: Took Help run ns-3 point four eight, interpret recorded output, populate the team report template, and analyze and insert the provided 1010 MHz default tone, 990 MHz retuning, and 900 MHz spectrum screenshots for questions one through three.

P.T.O